SoSConfigurator

Interface Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document consolidates the system interface designs. Detailed Java service/transport IDDs are retained as separate appendices, followed by available Go and Rust interface descriptions. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-SoSConfigurator-java.

Java architecture and responsibilities

SoSConfigurator is a Spring Boot support system with its own Arrowhead identity and registered sosConfiguration service (1.0.0), generalManagement and sysMonitor publisher. The current source supersedes the older README paragraph that calls it an unregistered operations tool. The Arrowhead connector defaults to 8494; a separate loopback operator console defaults to 8495. The catalog defines permitted clouds, consumers, exports, organizations and relay membership. PolicyEngine evaluates security and authorization rules before Planner creates relay-check, cloud-check, neighbours, grants, verify and optional discover steps. ColonyOS dispatches work to relay and per-cloud executors. Executors reevaluate the request with their own catalog before running the two owned step scripts. MySQL persists submissions and operational logs; ColonyOS retains workflow execution state. A successful submission is asynchronous acceptance, not confirmation that all clouds were configured. Execution across clouds is not an atomic database transaction. Inspect the workflow and reconcile partial effects before retrying. The Arrowhead connector uses mutual TLS and certificate-bound identity tokens. SosAccessFilter requires Sysop, an explicit sos.api.whitelist entry or a ConsumerAuthorization grant for the requested sosConfiguration operation. The console uses password sessions, Host/Origin checks and a separate loopback connector. Console-only routes return 404 on the Arrowhead connector.

Java interface contract

- GET /api/catalog: 200, catalog JSON.

- GET /api/status: 200, status JSON.

- POST /api/plan: JSON SosRequest; 200, Plan including allowed, violations and steps. A refused plan is still a 200 plan response.

- POST /api/deployments: SosRequest; 202 with {plan,id} when submitted; 422 with {error,plan} when policy refuses; 502 with {error} for ColonyOS I/O failure.

- GET /api/deployments: 200, submission list.

- GET /api/deployments/{id}: 200, deployment JSON; an id not matching 64 lowercase hexadecimal characters gives 404 with {error}; backend I/O failure gives 502.

- POST /sosconfigurator/general/mgmt/logs: optional LogRequestDTO; LogEntryListResponseDTO. GET /sosconfigurator/general/mgmt/get-config: optional repeated keys; KeyValuesDTO. These are management interfaces.

- POST /api/session is console-only; the cookie is ah_sos_console. It is not an Arrowhead identity-token login.

SosRequest fields: name (string), clouds (array of strings), exchanges (array of Exchange), verifyDiscovery (boolean). Exchange fields: consumerCloud, consumer, providerCloud, provider, service (strings), operations (array of strings). Null clouds/exchanges are treated as empty by the model helpers; the policy engine determines validity. Plan fields: sos, members, allowed, violations, steps, notes, evaluation. A Step contains kind, role and

target. Parsing errors or empty bodies give 400. Request bodies above 65536 bytes give 413. JSON responses from SosConfigurationAPI set Cache-Control: no-store. Authentication and authorization filters can reject earlier with 401/403. Unspecified framework errors are not assigned invented status guarantees here.

Java verification and source traceability

Checked against SoSConfiguratorConstants.java, SoSConfiguratorSystemInfo.java, SosConfigurationAPI.java, GeneralManagementAPI.java, SosRequest.java, Planner.java, SosAccessFilter.java and console/ConsoleFilter.java in the primary Java folder. These source snapshots are included. This documentation task did not execute workflows or change clouds.

Go implementation

No Go implementation folder for SoSConfigurator was found in the designated primary Go tree on 2026-11-09. No executable, build command, endpoint or feature parity is asserted. A future port must define architecture, state migration, service registration, DTO validation, authorization and transport coverage before it can replace this system.

Rust implementation

No Rust implementation folder for SoSConfigurator was found in the designated primary Rust tree on 2026-11-09. No executable, build command, endpoint or feature parity is asserted. A future port must define architecture, state migration, service registration, DTO validation, authorization and transport coverage before it can replace this system.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

No prior system design/IDD baseline existed; the design and contract above were derived from current primary sources. The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.